

EE492 - BTP 2:

Zephyr RTOS
on RPi-Pico for
Real-time applications

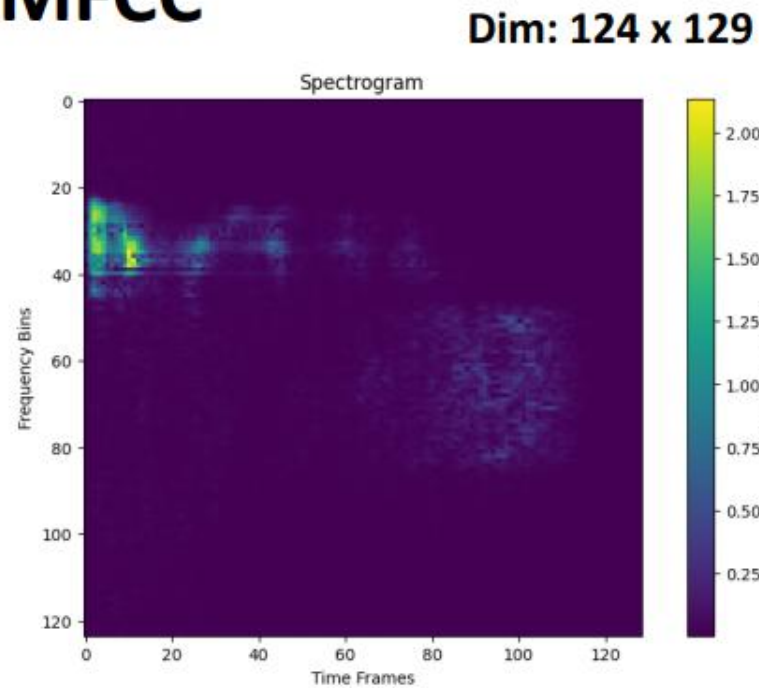
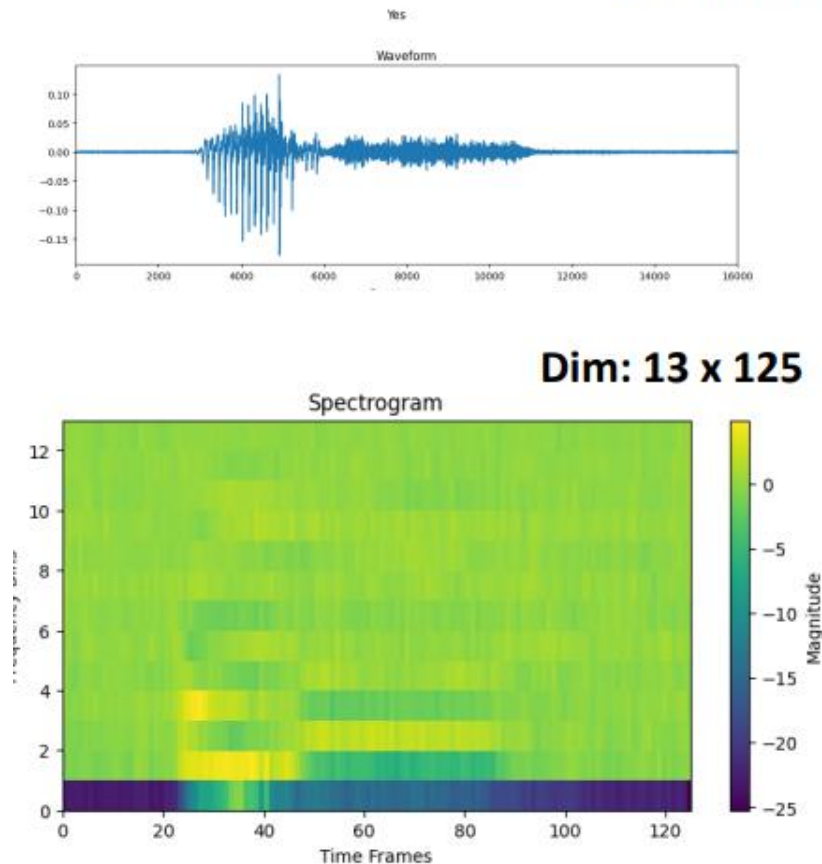
K P Lakshmeesh (22B1246)
EE BTech, IIT Bombay

Guides:
Prof. Siddharth Tallur
Prof. Srikanth Raj Chetupalli

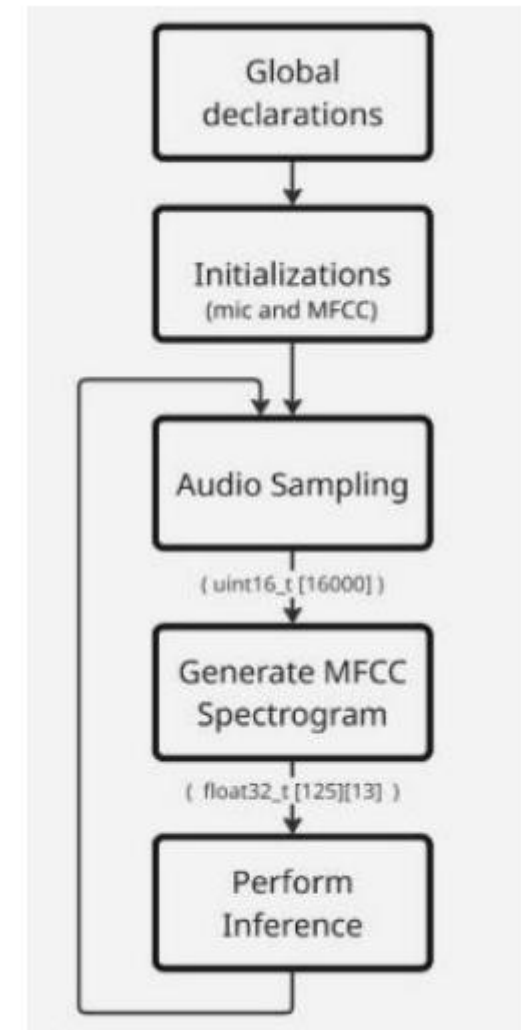
What was done in BTP-1?

- Explore and familiarise Zephyr RTOS and Tensorflow for microcontrollers (TFLM)
- Tested wifi, ADC interfacing
- Speech keyword detection application
 - Referred tensorflow's example using STFT input and 2D CNN model, and used their speech dataset for 8 keywords
 - MFCC spectrogram generation
 - 1D CNN model

STFT vs MFCC



Model	#params	Size	accuracy
STFT input 2D CNN	20K	78KB	77%
MFCC input 1D CNN	15.8K	62KB	82.1



Objectives for BTP-2

- Fix keyword detection application memory bug and improve model accuracy
- Explore DMA, SMP, overclocking
- PC streaming and shell interface
- Speech enhancement model deployment

1. Keyword detection application

- Memory related bugs (eg: MFCC stuck at 122nd frame ; audio sampling code order)
- Doubt on CMSIS-DSP library; created custom kissFFT based STFT library

FFT Output:

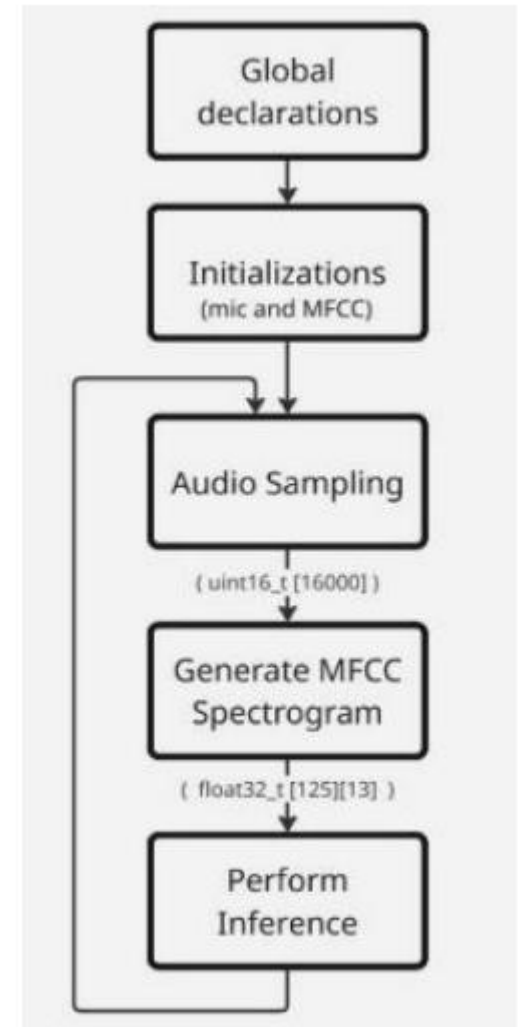
index 0:	Real: 0.003599,	Imag: 0.000000
index 1:	Real: 0.003737,	Imag: 0.086650
index 2:	Real: -0.404432,	Imag: -128.031555
index 3:	Real: 0.002855,	Imag: -0.155571
index 4:	Real: 0.003047,	Imag: -0.086439
index 5:	Real: 0.003106,	Imag: -0.061723
index 6:	Real: 0.003134,	Imag: -0.048582
index 7:	Real: 0.003148,	Imag: -0.040281

(for sine wave)

```
// Initializer
void stft_init(stft_lkp_cfg *stft_cfg, int fft_size, int hop_size,
              int sample_rate, float window[], char fft_cfg_buffer[], size_t *fft_cfg_buffer_len);

// STFT computation
void stft(stft_lkp_cfg *stft_cfg, const int *samples, float *output);

//deallocate buffers
void stft_free(stft_lkp_cfg *stft_cfg);
```



1. Keyword detection application

- Error from unaligned C-array model. Used alignas(8) to fix it.
TFLM uses flat-buffers to tightly pack the model and save space.
Pointer based parsing from base address.
- Further issues:
 - Very bad confidence on pico
 - MFCC mismatch between CMSIS and tensorflow:
 - 1.33 factor added to fix.
 - Microphone sampling delay issue
 - Thought it was not suitable for 16khz initially (Showed ~99us);
due to printf() ISR

```
for (int i=0; i<5; i++){
    start_cycles = k_cycle_get_32();
    int temp = read_mic();
    int delay = k_cyc_to_us_floor32((uint64_t)(k_cycle_get_32() - start_cycles));
    printf("delay [%d]: %d\n", i, delay);
    accu += delay;
}
```

- Temporary fix using interpolation
- Actually ~44us;
minor fluctuations fixed by busy wait
- DMA as best solution

```
for (uint16_t i=0; i<iters; i=i+2){
    next_deadline += period_cyc;
    out[i] = read_mic();
    while(k_cycle_get_32() < next_deadline){
    }
}
```

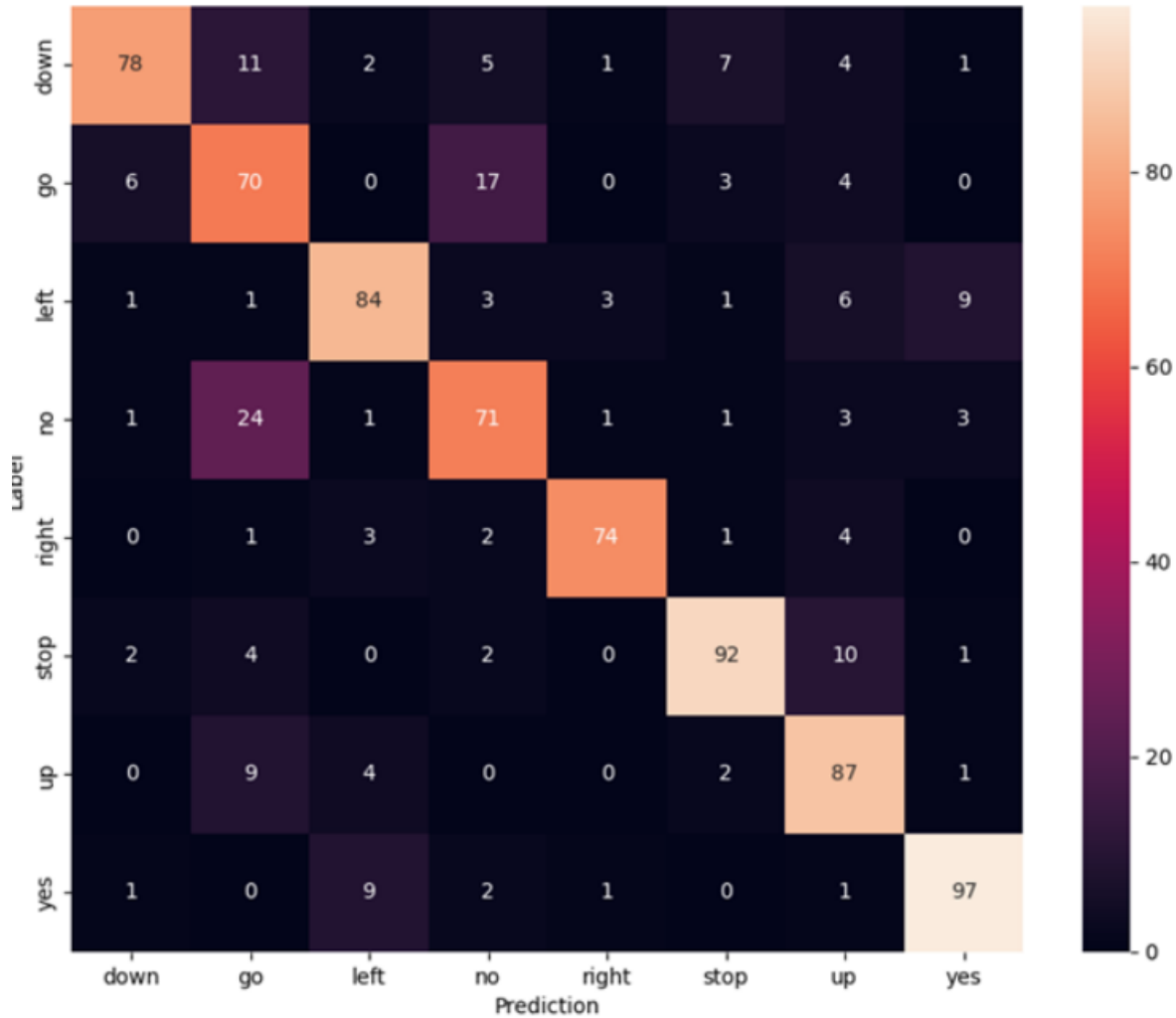
```
mfcc init success
-39.179688 2.266348 3.102156 2.0644
-33.260139 -1.816308 1.803770 1.589
-36.289978 -1.973216 1.142675 1.345
-39.212376 -1.905266 1.159501 1.287
-39.768543 -1.126074 1.678679 1.781
-40.839840 -1.272381 1.269942 1.806
-42.610706 -1.550953 1.598121 1.208
-42.517048 -0.689716 1.210146 1.469
-43.901436 -1.139096 1.410949 1.358
```

```
-45.454582, 3.0457134, 4.139763, 1.6827891, 0.123530
-40.329205, -2.5629926, 2.8307283, 0.6279178, -1.529
-43.96379, -2.8254113, 2.182528, 1.0426706, -0.65238
-46.1494, -2.3147147, 2.0082788, 0.8766454, -0.48100
-47.18337, -1.3614181, 3.4161, 1.7657356, -0.5003592
-47.54618, -1.5176327, 2.4192789, 1.5515323, -0.3353
-49.94876, -1.8100356, 2.4514635, 0.6463557, -0.3500
-49.484142, -1.0509032, 1.9128613, 0.9322042, -0.772
```

1. Keyword detection application

- Model retraining:

Older model:



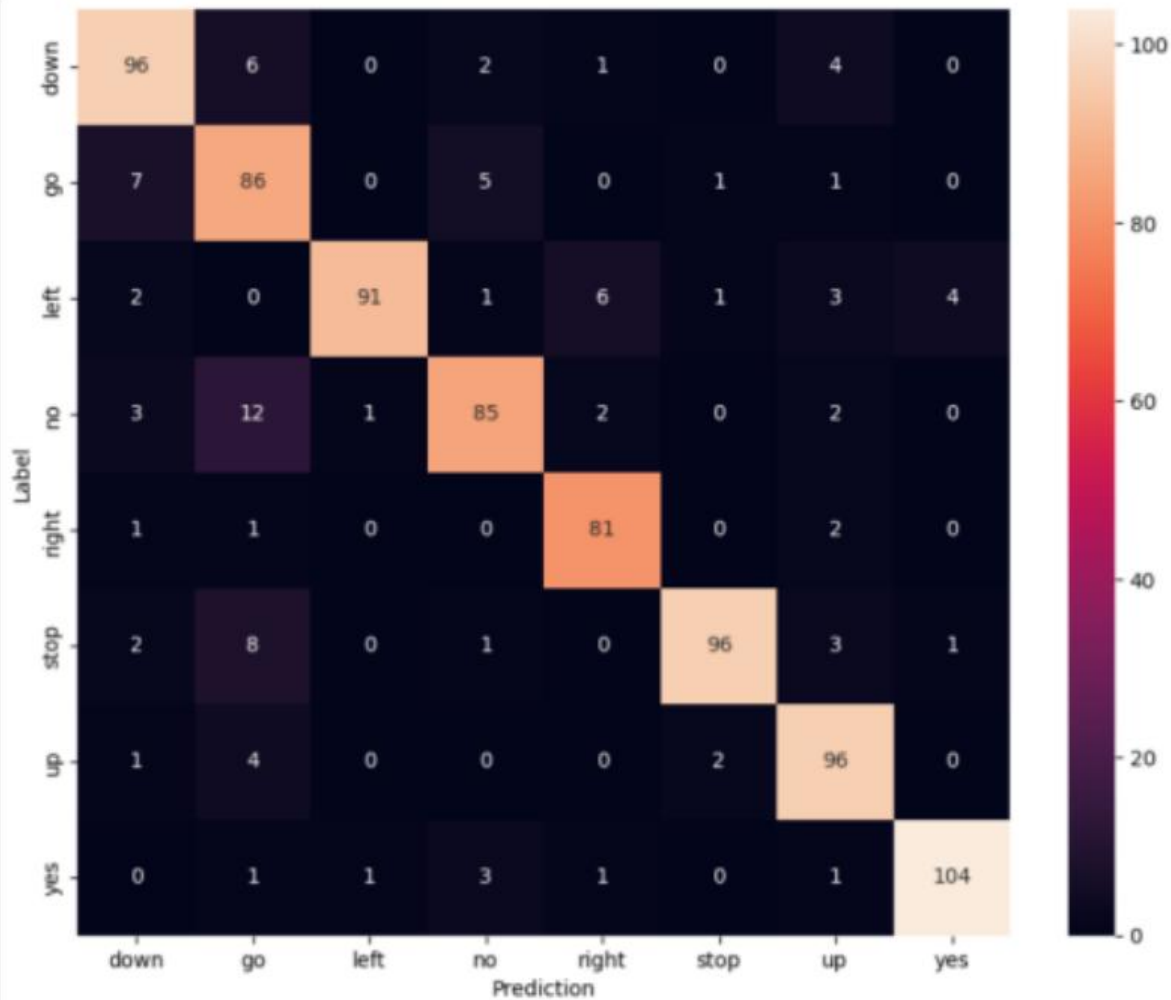
```
# model for mfccs as input (1D CNN)
model = models.Sequential([
    layers.Conv1D(8, 3, activation='relu', input_shape=(125, 13)),
    layers.MaxPooling1D(2),
    layers.Conv1D(16, 3, activation='relu'),
    layers.MaxPooling1D(2),
    #layers.Dropout(0.3), # (adding dropout caused accuracy to decrease)
    layers.Flatten(),
    layers.Dense(32, activation='relu'),
    layers.Dense(num_labels, activation='softmax')
])
```

82.1 validation accuracy

1. Keyword detection application

- Model retraining:

newer model:



```
#improved version using avg pooling instead of flattening + additional conv layer  
model = models.Sequential([  
    layers.Conv1D(16, 4, activation='relu', input_shape=(125, 13)),  
    layers.MaxPooling1D(2),  
    layers.Conv1D(32, 4, activation='relu'),  
    layers.MaxPooling1D(2),  
    layers.Conv1D(64, 4, activation='relu'),  
    layers.MaxPooling1D(2),  
    layers.GlobalAveragePooling1D(),  
    layers.Dense(64, activation='relu'),  
    layers.Dropout(0.3),  
    layers.Dense(num_labels, activation='softmax')  
)
```

88.3 validation accuracy

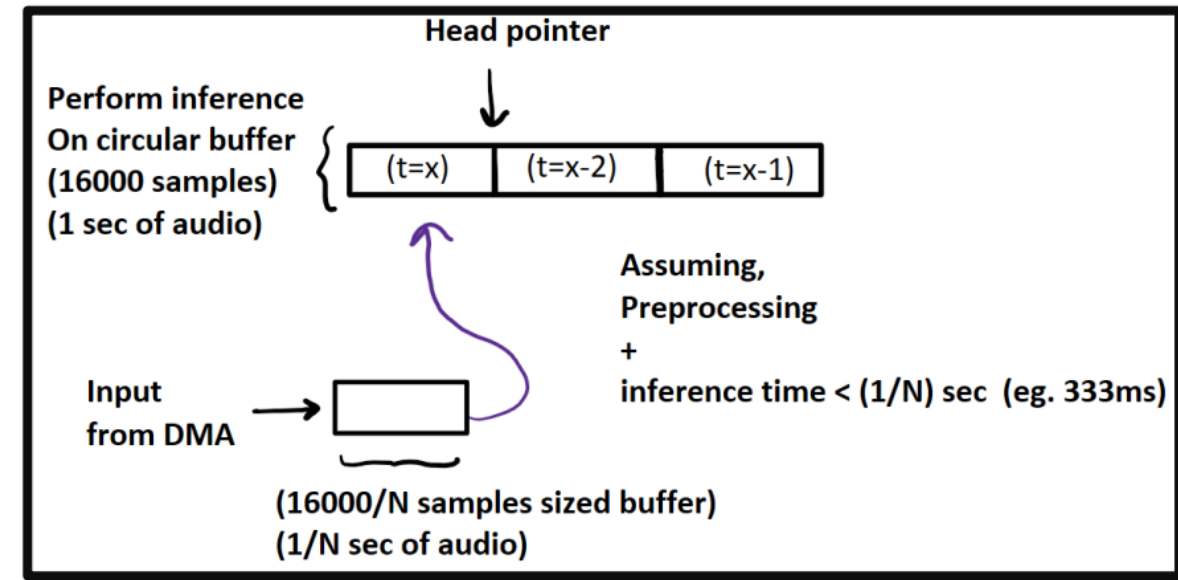
1. Keyword detection application

- Model retraining:
 - Tried increasing no. of DCT coeffs:

# DCT coefficients	Validation accuracy (over 5 runs)	#params	Spectrogram space
13	~0.788	15.86KB	52KB
16	~0.786	15.9KB	64KB
20	~0.776	16.0KB	80KB
26	~0.792	16.17KB	104KB
40	~0.792	16.5KB	160KB

- Only occupies ~22% of pico's RAM
- Other improvements:
 - Streaming into TensorArena instead of feeding spectrogram as a whole
 - Overclocking makes it possible for real-time.
(25% overlap for 2x speedup)

Task 1	Audio sampling	1 sec
Task 2	Spectrogram generation	~1.36 sec
Task 3	TFLM inference	~0.1 sec



1. Keyword detection application

- Results:

- performed well for 'go', 'no', 'right', 'up', and well differentiated. However, it was hard to detect 'down', 'stop', 'left' 'yes'.
- For eg, 'yes' gets detected as 'no' frequently.

The model was also only detecting it by playing the training dataset audio into the mic, suggesting possible overfittings.

keyword	Inference confidence
No	~0.757
Go	~0.777
Right	~0.789
up	~0.758

2. Zephyr Exploratory work

- Multi-threading
- SMP in pico
 - SMP means that multiple cores are capable of running Zephyr threads.
 - `arch_swap()` vs `arch_switch()`:
 - `Arch_swap()` was made for single core.
During context switch the pre-decided thread is loaded and next thread is decided.
Cross CPU synchronization has to be added; hard to do for all supported architecture in assembly
 - `Arch_switch()` decides next thread right at context switch
Minimise use of custom assembly. Do most work in `z_swap()` itself.
 - 60% speedup for cortex-M
 - Some more work left (CMSIS API migration instead of assembly, ARM TrustZone support, etc)
 - As per Zephyr's release roadmap, SMP support for pico's RP2040 is scheduled for October'26
 - Limitation for cortex M0+:
 - Higher Cortex-M (M3 onwards) have several features which make it easier to implement SMP like atomic LDREX/STREX.
 - Thus, implementation is not as straightforward
- Alternate for Multicore processing:
 - Compile Core 1 binary using pico SDK and store as array; write into core1 from zephyr in core0 via SIO fifo (specific launch sequence); limitations in synchronisation

```
*** Booting Zephyr OS build v4.2
Main thread started
T1 thread created
T2 thread created
Main thread is running
Thread 1 is running
Thread 2 is running
Thread 2 is running
Main thread is running
Thread 1 is running
Thread 2 is running
Thread 2 is running
```

3. DMA support on pico

- Why DMA?
- Pico supports M2M, P2M, M2P transfer.
- Upto max ADC rate (500KHz); max rate from zephyr api = ~22.2KHz
- DMA is not fully portable
 - Different MCUs have vastly different mechanisms to trigger and use DMA.
 - Zephyr cannot safely choose on behalf of the application
 - Possible but huge complexity overhead; full portability done on PC's. convenience and overhead at the cost of portability.

```
enum dma_channel_direction {  
    /** Memory to memory */  
    MEMORY_TO_MEMORY = 0x0,  
    /** Memory to peripheral */  
    MEMORY_TO_PERIPHERAL,  
    /** Peripheral to memory */  
    PERIPHERAL_TO_MEMORY,  
    /** Peripheral to peripheral */  
    PERIPHERAL_TO_PERIPHERAL,  
    /** Host to memory */  
    HOST_TO_MEMORY,  
    /** Memory to host */  
    MEMORY_TO_HOST,  
};
```

- M2M transfer test:

```
*** Booting Zephyr OS build v4
dst[0] = 0xABCDABCD
dst[1] = 0xABCDABCD
dst[2] = 0xABCDABCD
dst[3] = 0xABCDABCD
dst[4] = 0xABCDABCD
dst[5] = 0xABCDABCD
dst[6] = 0xABCDABCD
dst[7] = 0xABCDABCD
dst[8] = 0xABCDABCD
dst[9] = 0xABCDABCD
```

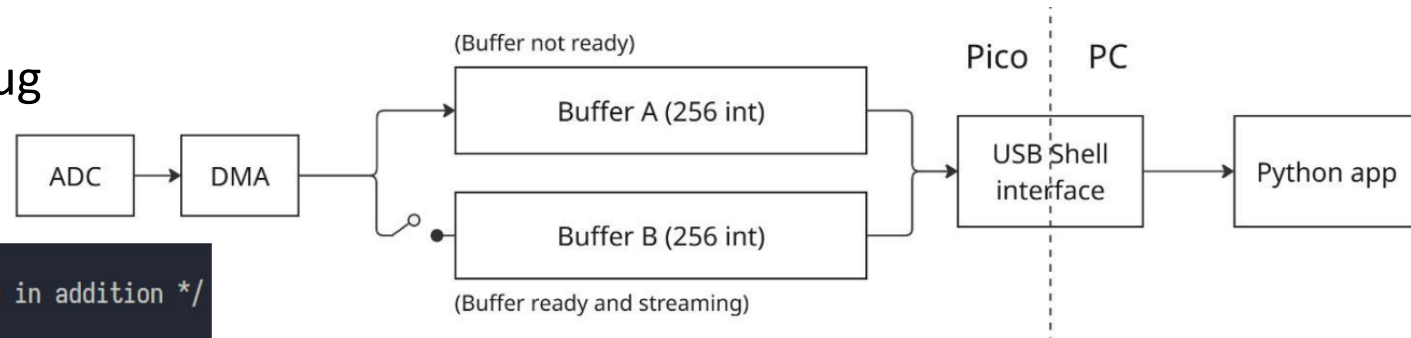
P2M transfer test (ADC) :

```
*** Booting Zephyr OS build v4.2.0-rc3-44-g366d45fd34
initial buffer:
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
buffer finally:
34747 3DMA block complete
4747 34747 34747 34747 34747 34747 34747 34747 34747
4747 34747 34747 34747 34747 34747 34747 34747 34747
4747 34747 34747 34747 34747 34747 34747 34747 34747
4747 34747 34747 34747 34747 34747 34747 34747 34747
4747 34747 34747 34747 34747 34747 34747 34747 34747
```

- P2M was hard to setup; check alternate OS options as well (apache nuttx, RT-thread)

4. PC streaming and shell interface

- Setup 2 COM port
 - COM3 for start/stop cmd and printk() debug
 - COM4 for data stream



```
&zephyr_udc0 {  
    /* cdc-acm-console snippet already makes a port, so we make one more in addition */  
    cdc_acm_lkp: cdc_acm_lkp {  
        compatible = "zephyr,cdc-acm-uart";  
    };  
};
```

- Baud rate:
 - CDC-ACM ignores baud rate settings
 - transfers data at the full speed for USB2.0 (12Mbps).
 - The baud rate settings mentioned in the zephyr drivers are only for legacy compatibility reasons, required during USB enumeration.
- For 16KHz, 16-bit samples, need atleast 320Kbps. standard baud rate 460800 chosen.

```
SHELL_CMD_REGISTER(start, NULL, "Start recording", cmd_start);  
SHELL_CMD_REGISTER(stop, NULL, "Stop recording", cmd_stop);  
SHELL_CMD_REGISTER(test_usb, NULL, "Test usb streaming", test_usb_cmd);
```

4. PC streaming and shell interface

- Issues:
 - DMA config ping-pong didn't work; (scatter gather) changed to reloading inside callback

Previously: (empty audio file)

```
block_a_cfg.source_address = 0x4004C00C; //ADC FIFO address
block_a_cfg.dest_address   = (uint32_t)buffer_a;
block_a_cfg.block_size     = sizeof(buffer_a);
block_a_cfg.source_addr_adj = DMA_ADDR_ADJ_NO_CHANGE;
block_a_cfg.dest_addr_adj   = DMA_ADDR_ADJ_INCREMENT;
block_a_cfg.next_block     = &block_b_cfg;

block_b_cfg.source_address = 0x4004C00C; //ADC FIFO address
block_b_cfg.dest_address   = (uint32_t)buffer_b;
block_b_cfg.block_size     = sizeof(buffer_b);
block_b_cfg.source_addr_adj = DMA_ADDR_ADJ_NO_CHANGE;
block_b_cfg.dest_addr_adj   = DMA_ADDR_ADJ_INCREMENT;
block_b_cfg.next_block     = &block_a_cfg;

dma_cfg.channel_direction = PERIPHERAL_TO_MEMORY;
dma_cfg.source_data_size  = 2;
dma_cfg.dest_data_size    = 2;
dma_cfg.block_count       = 2;
dma_cfg.head_block        = &block_a_cfg;
dma_cfg.dma_slot          = 0x0b; //related to dreq
// dma_cfg.dma_slot        = RPI_PICO_DMA_SLOT_ADC;
dma_cfg.dma_callback       = dma_callback;
dma_cfg.complete_callback_en = 1; //fires callback after each block
```

Now: (non-empty files)

```
block_a_cfg.source_address = 0x4004C00C; // ADC FIFO address
block_a_cfg.dest_address   = (uint32_t)buffer_a;
block_a_cfg.block_size     = sizeof(buffer_a);
block_a_cfg.source_addr_adj = DMA_ADDR_ADJ_NO_CHANGE;
block_a_cfg.dest_addr_adj   = DMA_ADDR_ADJ_INCREMENT;
block_a_cfg.next_block     = NULL;

dma_cfg.channel_direction = PERIPHERAL_TO_MEMORY;
dma_cfg.source_data_size  = 2;
dma_cfg.dest_data_size    = 2;
dma_cfg.block_count       = 1;
dma_cfg.head_block        = &block_a_cfg;
dma_cfg.dma_slot          = 0x0b;
dma_cfg.dma_callback       = dma_callback2;

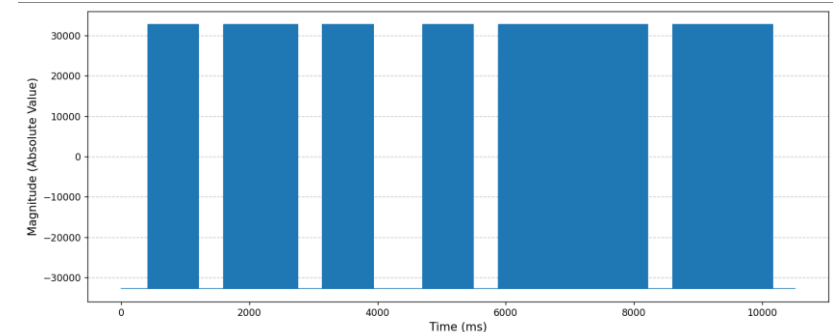
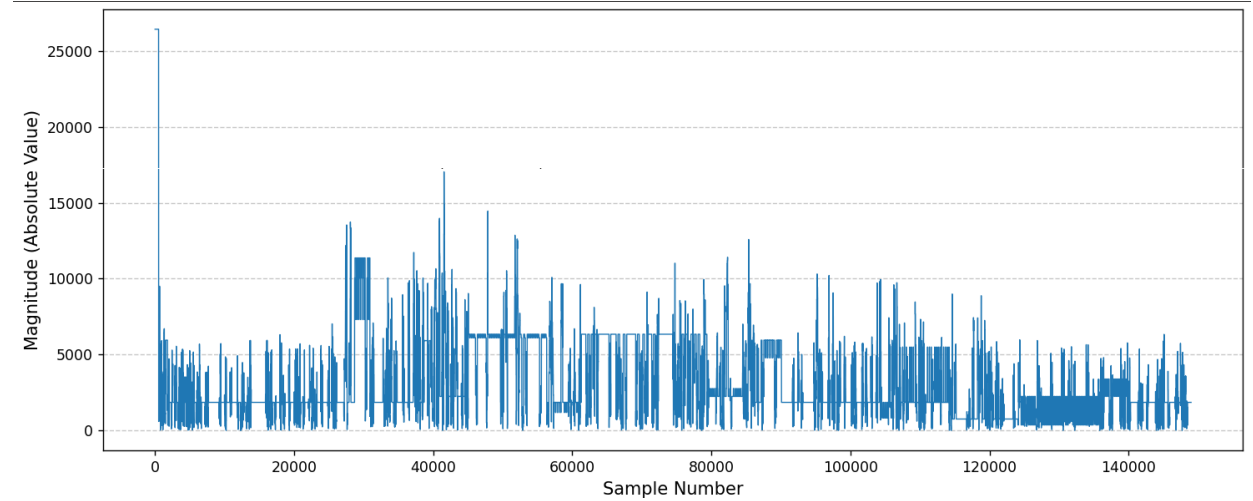
void dma_callback2(const struct device *dev, void *user_data, uint32_t channel, int status){
    if(!buffer_a_filled){
        dma_reload(dev, channel, 0x4004C00C, (uint32_t)buffer_b, sizeof(buffer_b));
        dma_start(dev, channel);
    }
}
```

4. PC streaming and shell interface

- Issues:
 - Incorrect audio duration
 - Speech is not audible after recording (rumbling sounds)
- Tests to fix:
 - USB: Send fixed bytes over USB

```
Enter command (start / stop / exit): [PICO LOG]: test_usb
[PICO LOG]: started test USB streaming (no adc or DMA used)
TEST READ: 10 bytes: b'\x01\x00\x02\x00\x03\x00\x04\x00\x05\x00'...
```

- Now enabled DMA:

[illegible]

5. Over-clocking

- Default pico clock rate = 133Mhz, overclocked to 250MHz.
- Test on STFT for 16K Samp:
 - 2x improvement for 1.88x clock-rate

Clock rate	Time taken
133Mhz	0.979 sec
250Mhz	0.489 sec

- Stable overclock upto 250-270MHz. Can be increased upto 360 MHz by increasing core voltage
- Modifying Zephyr overlay did not affect the voltage regulator.
Non-portable workaround is using `vreg_set_voltage()` from picoSDK.

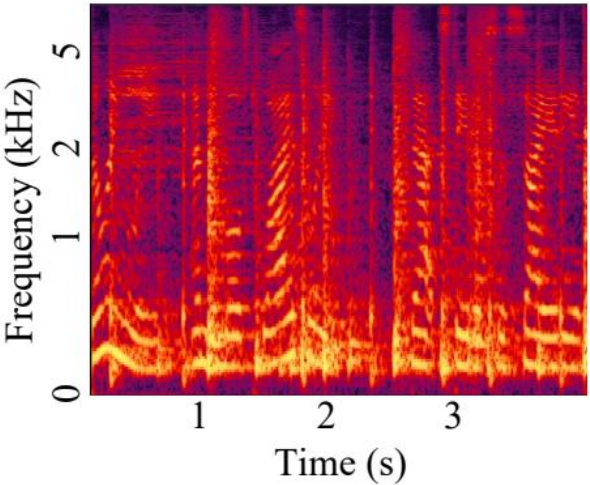
```
CONFIG_SYS_CLOCK_HW_CYCLES_PER_SEC=250000000
```


6. Speech enhancement

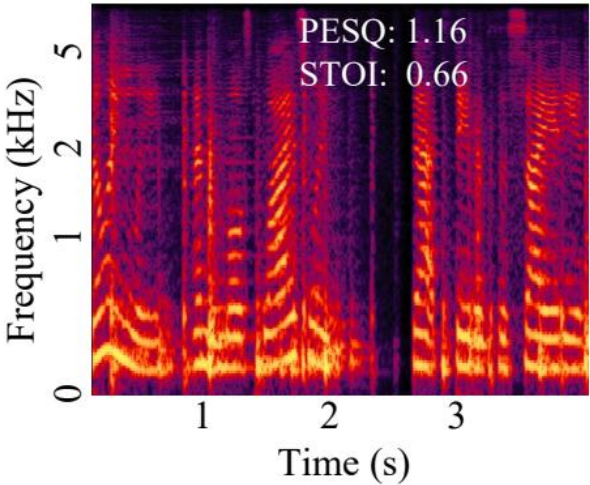
- Deploying GTCRN on pico
 - Very low parameters (23.7K) and much better performance than other models

Table 2: Performance on VCTK-DEMAND test set.

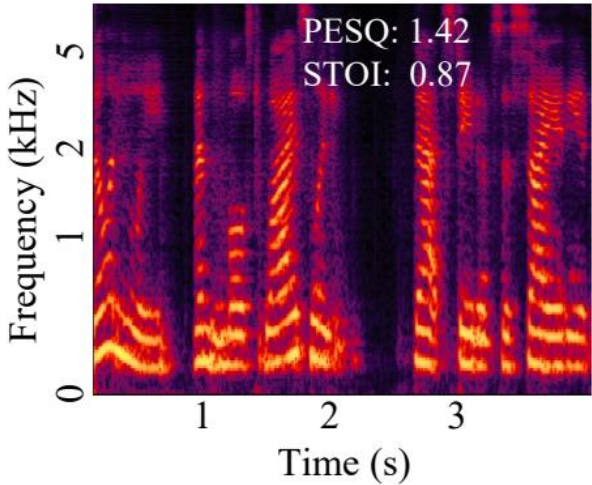
	Para. (M)	MACs (G/s)	SISNR	PESQ	STOI
Noisy	-	-	8.45	1.97	0.921
RNNoise (2018)	0.06	0.04	-	2.29	-
PercepNet (2020)	8.00	0.80	-	2.73	-
DeepFilterNet (2022)	1.80	0.35	16.63	2.81	0.942
S-DCCRN (2022)	2.34	-	-	2.84	0.940
GTCRN (proposed)	0.02	0.04	18.83	2.87	0.940



(a) Noisy



(b) Enhanced by RNNoise



(c) Enhanced by GTCRN

6. Speech enhancement

- Converter issues:
 - Started with pre-built .onnx model.
 - Several lengthy model conversion errors (dimension mismatch from GRU; library/version dependency)
 - Onnx2tf, onnx/onnx-tf libraries, emmetrix onnx-to-c Converter
 - Pytorch to tflite conversion (using LiteRT)
 - Compiler cant do grouped convolution
 - Split into 2 convolutions
 - Error in ConvBlock() due to upsampling for stride>1.
 - Reimplemented using supported nn.conv2D
 - Successfully converted to .tflite (1685KB) ; expectation after quantizing = ~420KB

6. Speech enhancement

- Quantisation issues:
 - Select_V2 node issue in streamEncoder;
 - Due to slicing cache; slice cache outside model and feed it in separately
- Model size issue after unrolling bidirectional GRU
(344KB to 1685KB on conversion from onnx to tflite; optimistic final size after quantizing = ~420KB)
 - Model c-array may fit easily in the ROM (2MB).
But constraint is on the tensor arena size in RAM (264KB).
Likely as 9000 bytes for keyword detection (30KB .tflite)
- Alternate solutions:
 - Shift to non-streaming
 - Cut down depth/width of convolutions
 - Use TCN instead of bidirectional GRU

6. Speech enhancement

- RNNoise:
 - Full quantized Tflite model available ; simplifies deployment
 - Relies heavily on both DSP and GRU
- Broad steps:
 - Biquad high pass filter (remove low freq noise)
 - Compute bark band energies, then DCT on log spectrum (22 BFCC)
 - Pitch search, fft, band energies, DCT (6 features)
 - If (not silent):
 - Compute RNN (input 42 features, output 22 band gain values)
 - Pitch filter
 - Frame synthesis
- Issues:
 - pitch filter seemed to be computationally expensive.
 - Many of pre-processor C code is PC specific. Requires rewriting for MCU.

Note:

other features are 1st and 2nd deri. of first 6 BFCCs.
Pitch period , spectral non-stationarity metric

Thank you!